Computer Networks – Application Layer

Assignment: Dual-Panel Activity & Protocol Visualizer

AI-Assisted Development (Choice of Agentic Platforms & Models)

Course / Subject	Computer Networks – Application Layer
Development Approach	AI-Agent assisted (student's choice of platform & model)
Duration	1–2 weeks (recommended)
Submission	Source code + AI usage log / artifacts + short demo video + reflection

1. Objective

Students will design and build a web dashboard that allows a user to perform three common application-layer activities (Browsing, Mail, and Streaming) while simultaneously visualizing the underlying protocol exchanges (DNS, HTTP, SMTP) in real time. The project must be developed with the help of an **agentic AI coding platform / model of the student's choice**. The goal is to connect the user's action with the actual messages exchanged at the application layer and to gain practical experience with modern AI-assisted software development.

2. Choice of Agentic AI Platforms & Models

Students are free to select any popular agentic AI coding platform or model. Examples include (but are not limited to):

- **Google Antigravity** (with Gemini models)
- **GitHub Copilot** / Copilot Workspace / Copilot Chat
- **Claude** (Claude Code, Claude.ai, or Anthropic API – Sonnet / Opus)
- **Cursor** (with Claude, GPT, or Gemini)
- **Gemini** (Google AI Studio, Gemini Code Assist, or Gemini API)
- **OpenAI** tools (ChatGPT, Codex, or o-series models)
- Any other reputable agentic coding assistant the student prefers

Students must clearly state in their submission **which platform(s) and model(s)** they used and provide evidence of AI assistance (prompt history, chat logs, agent artifacts, or screenshots).

3. Dashboard Layout (Strict Requirement)

The application must contain **exactly two main panels** placed side-by-side (or top-bottom on smaller screens):

Left Panel – Activity Panel

The user can choose and perform one of the following actions:

- **Browsing**: Enter a URL and "visit" a page (simulated fetch is acceptable).
- **Mail**: Compose and "send" a simple email (fields: To, Subject, Body).
- **Streaming**: Start a simulated video stream (play / pause and quality selector).

The left panel must show clear UI controls, status messages, and a short activity log for the selected action.

Right Panel – Protocol Visualization Panel

As soon as the user performs an activity on the left, the right panel must **simultaneously** display a step-by-step, animated or sequential visualization of the relevant protocols:

Activity	Protocols to Visualize in Parallel

Browsing	DNS → HTTP (request + response)
Mail	DNS (optional) → SMTP conversation
Streaming	DNS → HTTP (manifest / playlist + segments)

For each protocol the visualization must show:

- Exact messages / commands (e.g., DNS query & response, HTTP GET + headers + status line, SMTP EHLO / MAIL FROM / RCPT TO / DATA / QUIT)
- Direction of each message (client → server or server → client)
- Sequence / timing indicators
- Key fields highlighted
- Simple animation or progressive reveal (arrows, timeline, or state-machine style)

Users must be able to pause, step forward/backward, or replay the visualization.

4. Technical & AI Requirements

- The project must be developed with substantial assistance from an agentic AI coding tool of the student's choice (see Section 2).
- Students are expected to use the AI for planning architecture, generating the dual-panel UI, implementing protocol simulators, creating live synchronization between panels, and debugging / iterating.
- Preferred technology stack: Python + FastAPI or Flask + HTML/JS. Streamlit or Gradio are also acceptable. A pure frontend (React/Vue) is allowed.
- Simulation of protocols is fully acceptable (real network sockets are optional extra credit). Visualizations must remain accurate with respect to real protocol behavior.
- Students must retain and submit evidence of AI usage (prompts, responses, agent artifacts, or screenshots).

5. Deliverables

1. Working dashboard (complete source code + clear instructions to run it).
2. AI usage log / artifacts (prompt history, chat exports, agent records, or screenshots) clearly showing the platform and model used.
3. Short demo video (2–4 minutes) or a set of screenshots showing all three activities with the right panel updating in parallel.
4. Reflection document (1–2 pages) covering:
   - Which AI platform(s) and model(s) were chosen and why
   - How the two panels stay synchronized
   - What the AI got wrong and how you corrected it
   - Key differences observed between DNS+HTTP, SMTP, and streaming HTTP flows

6. Evaluation Criteria

Criterion	Weight
Correct dual-panel layout and live synchronization between activity and protocol panels	35%
Accuracy of DNS / HTTP / SMTP message visualization	25%
Effective use of chosen AI platform/model (prompt quality, iteration, evidence)	25%
Code quality, UI polish, and reflection document	15%

7. Suggested Starter Prompt (Adapt to Your Chosen AI Tool)

You may adapt the following prompt for Antigravity, Cursor, Claude, Copilot, Gemini, or any other tool:

```
Create a dual-panel web dashboard in Python (FastAPI + HTML/JS or Streamlit).

Left panel: Activity selector with three modes –
• Browsing (URL input + Visit button)
• Mail (compose form: To, Subject, Body + Send button)
• Streaming (Play / Pause + quality selector)

Right panel: Live protocol visualizer that updates in parallel with the left panel:
• Browsing → show DNS query/response followed by full HTTP request/response
• Mail → show complete SMTP conversation (EHLO, MAIL FROM, RCPT TO, DATA, QUIT)
• Streaming → show DNS + HTTP playlist/manifest request + segment requests

Make the right panel animated or step-by-step (pause / next / previous / replay). Keep both
panels synchronized so that every user action on the left immediately triggers the
corresponding protocol visualization on the right.
```

8. Important Notes for Students

- Focus on correctness of protocol message sequences more than fancy graphics.
- Clearly document the AI platform and model you used; this is part of the evaluation.
- If the AI generates incorrect protocol behavior, you must correct it and explain the correction in your reflection.
- Extra credit ideas: real socket-based servers, basic TLS indication, comparison of persistent vs non-persistent connections, or using multiple AI tools and comparing their effectiveness.

This assignment bridges the gap between what a user does and what actually happens at the application layer, while giving hands-on experience with modern AI-agent development using a platform of your choice.

Good luck!